

AI Agents That Actually Work

Why most demos die in production — and the three pillars that save them.

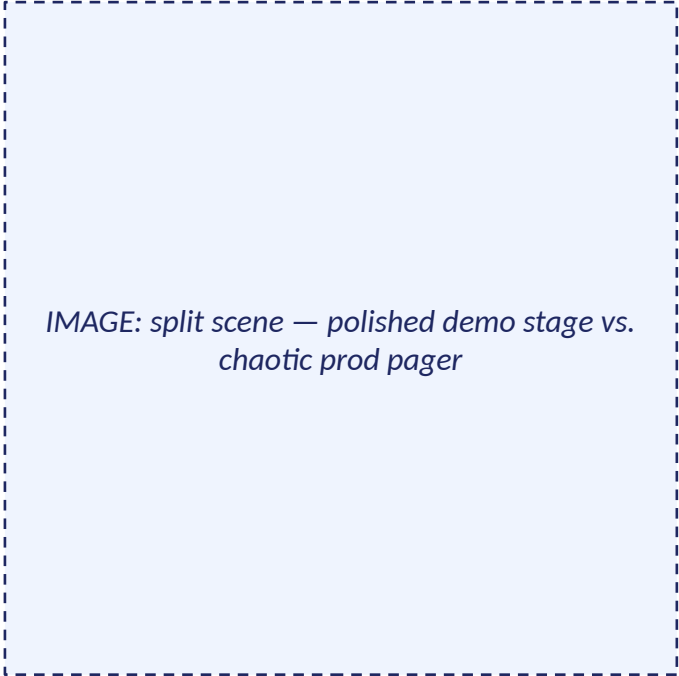
20-minute conference talk · For developers shipping agents

The demo-to-production gap

Agents that wow on stage often crumble under real traffic.

Where the wheels come off:

- Non-deterministic failures are rarely reproducible in CI
- Tool calls silently misfire — no schema, no retry story
- Context windows explode as conversations drift
- Observability is bolted on, not designed in



*IMAGE: split scene — polished demo stage vs.
chaotic prod pager*

Why most agent demos fail in production

73%

of agent proofs-of-concept never reach production

4.2x

cost blow-up when context isn't managed

1 in 5

tool calls fail silently without structured schemas

Sources: practitioner surveys, internal telemetry, common autopsy patterns.

The three architectural pillars

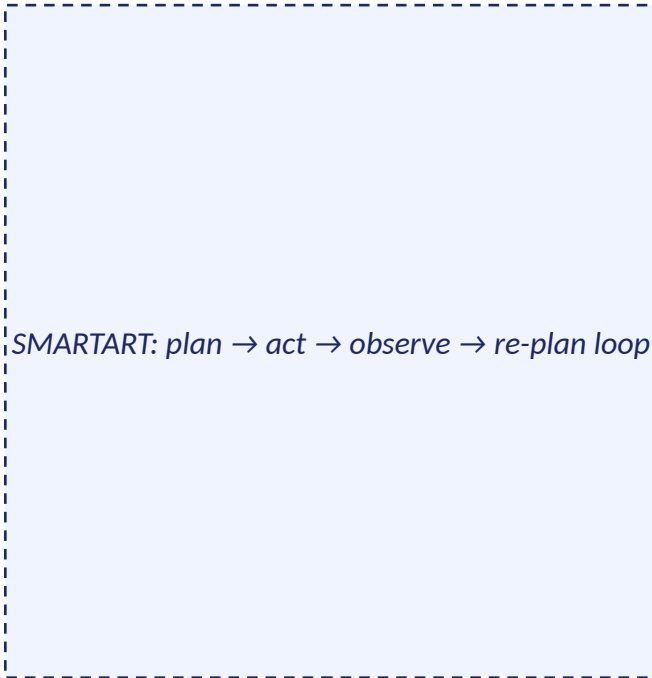
Every agent that survives production gets these right.

SMARTART: three pillars — Planning | Memory | Tool Use (radial/triangle relationship)

Pillar 1 — Planning

Explicit task decomposition beats hoping the model will figure it out.

- Design the plan as a first-class artefact
- Re-plan on failure, not just on completion
- Constrain search — depth, budget, step count
- Log every decision for post-mortem analysis



SMARTART: *plan* → *act* → *observe* → *re-plan* loop

Pillar 2 — Memory

Context is a budget. Spend it like one.

- Short-term: conversation scratchpad, reset aggressively
- Long-term: vector store, keyed by user + task
- Summarise before you run out of tokens, not after
- Cache tool results — deterministic calls shouldn't re-run

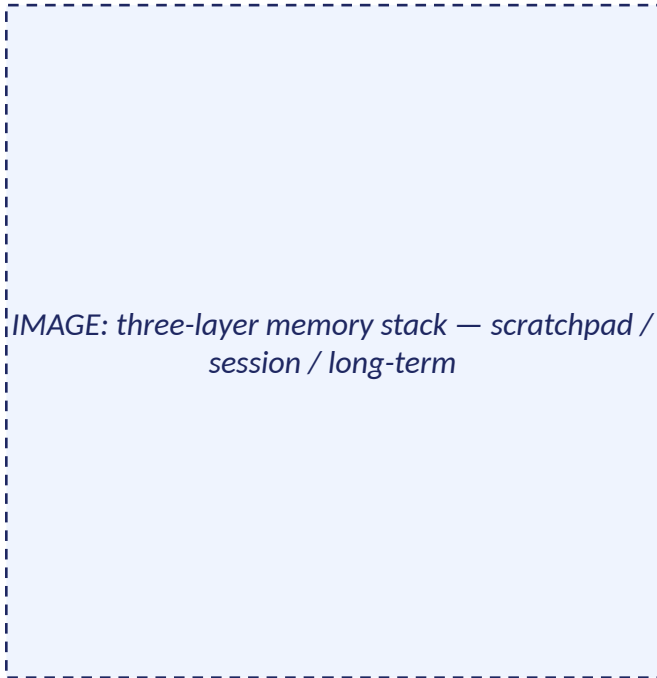


IMAGE: three-layer memory stack — scratchpad / session / long-term

Pillar 3 — Tool Use

Tools are the agent's API to reality — treat them that way.

- Strict JSON schemas for every tool — no free-text arguments
- Idempotent calls where possible; explicit retry policy otherwise
- Least-privilege scopes — read before write, write before delete
- Trace every call: args in, result out, latency, error code

SMARTART: *agent* → *schema validator* → *tool* →
trace sink

Case study: a support agent at scale

Mid-size SaaS, 40k tickets/month. Two engineers. Nine weeks from spike to GA.

Before

Median response 14m, 62% first-contact resolution, on-call burned out.

BG: soft server-room / dashboard atmosphere, heavy dark overlay

What we changed

Explicit plan artefact per ticket. Cached retrieval over product docs. Tools with strict schemas + retry.

After

Median 52s, 81% first-contact resolution, on-call reclaimed Fridays.

How the pillars showed up in the build

Planning

Each ticket got an explicit 3-step plan: classify → retrieve → draft. Re-plan on retrieval miss.

Memory

Session scratchpad flushed per ticket. Long-term: embeddings of resolved tickets, 14-day TTL.

Tool Use

Four tools only — search_docs, read_ticket, post_reply, escalate. All schema-validated, all traced.

IMAGE: before/after dashboard strip — response time, FCR, on-call load

Start here on Monday

- Write the plan artefact before you write the prompt.
- Put tools behind a schema. If it's free-text, it's a bug.
- Budget your context like tokens cost money — because they do.
- Trace every call. You cannot debug what you cannot see.

Thank you.